



# Instrucciones generales del parcial

- **Debes usar exclusivamente** las implementaciones base de Pila y Cola que te proveo.
- Todas tus funciones deben incluir **TypeHints**.
- Para la **Cola de Prioridad**, si el ejercicio la requiere, debes programarla tú (puedes implementarla con listas de Python).
- **Prohibido** usar cualquier otra colección (listas, sets, diccionarios, arrays dinámicos, etc.) **como estructura de solución para los ejercicios del parcial**.
  - Nota: En algunos ejercicios **se recibirán strings o listas como parámetros**, pero **toda la lógica** debe basarse **solo** en **pila, cola o cola de prioridad**, según el enunciado.
- Si modificas una estructura durante el proceso, **debes dejarla en su estado original** al finalizar (salvo que el enunciado diga lo contrario).
- Escribe **funciones puras** y **bien documentadas**; evita Entrada/Salida en consola salvo para pruebas.
- Todos los parciales están sujetos a sustentación. Si se detecta indicios de fraude, será citado a sustentación.
- Cualquier intento de fraude anulará el parcial.

## 1) Pila vs Cola: ¿mismos valores y mismo orden?

**Objetivo.** Implementa una función que determine si **una cola** y **una pila** contienen **exactamente los mismos valores en el mismo orden lógico en su posición inicial, posición de la mitad y posición final**, independientemente de su tamaño.

**Definición de “mismo orden lógico”:**

- Para la **cola**, el orden es **frente** → **fondo**.
- Para la **pila**, el orden es **base** → **tope**.

Es decir, si lees la cola de frente a fondo y la pila de base a tope, deben producir la **misma secuencia para estos tres valores, independientemente de su longitud**.

**Restricciones.**

- Usa **solo** las operaciones permitidas por las implementaciones base.
- **Debes restaurar** la pila y la cola a su **estado original**.
- **No** puedes copiar elementos a estructuras no permitidas.

**Ejemplo (conceptual).**

- Cola (frente→fondo): 1, 3, 5, 7, 11, 13, 17
- Pila (base→tope): 1, 7, 17 → **True** (porque el primero de ambas es el 1, el del medio de ambas es 7 y el último de ambas es 17)
- Nota: Si la pila es (tope hacia arriba) 17, 7, 1, su **base→tope** es 1, 7, 17.

## 2) ¿Es palíndromo? (puedes usar pila y cola)

**Objetivo.** Implementa una función que determine si un **string** es **palíndromo**.

**Normalización (obligatoria).**

- Ignora **espacios**, **tildes** y **signos de puntuación**.
- No distingas entre **mayúsculas/minúsculas**.

**Restricciones.**

- Puedes usar **pila** y **cola** (ambas o una, como prefieras).
- Toda la verificación debe basarse en estas estructuras.
- Nada de slicing ni estructuras prohibidas para comparar.

**Ejemplos.**

- "Anita lava la tina" → True
- "A man, a plan, a canal: Panama!" → True
- "Hola" → False

### 3) Centro de emergencias médicas: prioridad por gravedad del paciente

**Contexto.** el sistema recibe **pacientes** que llegan a urgencias. La prioridad depende del nivel de gravedad:

1. **Crítico** 
2. **Urgente** 
3. **Moderado** 

#### Reglas de atención.

- Atiende siempre primero a los críticos; si no hay, a los urgentes; si no hay, a los moderados.
- **Estabilidad (FIFO por prioridad):** dentro de la misma gravedad, se atiende en orden de llegada.

#### Restricciones.

- Implementa la cola de prioridad **con listas de Python** (no `heapq`, no `PriorityQueue`).
- Debes devolver la cola de prioridad con el orden correcto de atención.
- Cada paciente llega como un registro o tupla con:  
`{"nombre": str, "gravedad": "crítico"|"urgente"|"moderado"}`

#### Ejemplo.

Entrada:

- ("Ana", "moderado")
- ("Luis", "crítico")
- ("Sofía", "urgente")

- ("Pedro", "crítico")
- ("María", "moderado")

Salida de atención:

Luis (crítico) → Pedro (crítico) → Sofía (urgente) → Ana (moderado) → María (moderado)